8 GenAI System Design Questions

AI Engineers Struggle to Answer in Interviews

Most candidates prepare theory.
But modern AI interviews focus on designing **real GenAI systems**.

Here are 8 system design questions that frequently appear in AI/ML interviews.

(Swipe to test yourself →)

Q1. How would you design the end-to-end architecture for a production GenAI application that serves thousands of users?

> **The key principle: never let the LLM be in the critical path of user-facing latency if you can avoid it, and never let a single component be a single point of failure.**

Here is how I would structure it:

Entry layer

API gateway handles auth, rate limiting, and request routing. Every request gets a unique trace ID here.

Orchestration layer

Stateless service decides the flow (generation, RAG, agent workflow). Builds prompts, calls LLM, assembles final response.

LLM access layer

LLM proxy handles retry logic, secondary fallbacks, cost tracking, and response caching. No direct LLM calls.

Data & Async layers

Vector store, PostGreSQL, Redis. Async job queues for long-running workflows with worker polling.

AI INTERVIEW KIT

All You Need: One Kit to Clear Your AI Interview

After seeing how AI interviews are evolving,
I created an AI Interview Kit.
It covers the areas companies actually test:

- AI fundamentals
- Transformers (in depth)
- LLM architectures
- Prompt engineering
- Fine-tuning
- RAG systems
- AI agents
- Memory systems
- Databases
- Backend for AI systems
- Deployment strategies
- GenAI system design

Everything structured to help you prepare for modern AI interviews.
Check the caption to get the AI Interview Kit.

Q2. How would you integrate multiple models (text, image, embedding) in a single application architecture?

Multi-model architectures need a unified request routing layer so application code does not need to know the specifics of each model's API.

Model abstraction layer

Create a unified model interface. Behind it, adapters handle differences:

```
Application → ModelRouter → TextAdapter (GPT-4o)
                          → ImageAdapter (SDXL/DALL-E)
                          → EmbedAdapter (text-emb-3)
```

→ **Routing:** The router routes requests to appropriate adapters, handles auth, and normalizes responses to a common interface.

→ **Unified observability:** Normalize token counts, image generation steps, and vector dimensions into a common cost unit for billing.

→ **Async handling:** Route slow image generation requests through async job queues. Text generation can remain synchronous.

→ **Failure isolation:** Keep models independent - circuit breakers, retry policies, and fallback options.

Bharath Reddy ⤵ Check the caption for AI interview Kit Repost

Q3. What architectural changes are needed when moving from prototype GenAI applications to production systems?

Most GenAI prototypes are built with direct API calls, in-memory state, and hard-coded prompts. Each is a production failure waiting to happen.

→ **Direct API calls → LLM proxy:** The proxy adds retries, fallbacks to secondary providers, rate limiting, cost tracking, and caching.

→ **In-memory state → persistent stores:** Move conversation history to DBs, sessions to Redis, and files to object storage.

→ **Synchronous calls → async jobs:** Move long document ingestion, agent workflows, and slow generations to async job queues.

→ **Hard-coded prompts → versioned registry:** Prompts need version control, A/B testing capability, and rollback support.

→ **No observability → full tracing:** Add trace IDs, per-stage timing, token counts, and model version logging.

→ **Single instance → horizontal scaling:** Ensure stateless application design to load balance across multiple nodes.

Q4. What bottlenecks typically appear when scaling RAG systems?

RAG has four distinct bottlenecks and they do not all appear at the same scale.

Embedding generation bottleneck (early scale)

API calls add latency and cost. **Solution:** Cache query embeddings, use batch API calls for ingestion.

Vector search latency bottleneck (medium scale)

ANN search latency increases with index size. **Solution:** HNSW parameter tuning, index sharding, read replicas.

LLM generation bottleneck (any scale)

The largest latency contributor. **Solution:** Streaming responses, smaller models for simple queries, semantic caching.

Reranking bottleneck (quality-focused)

Cross-encoders add 200-500ms latency. **Solution:** Apply reranking only to queries above a complexity threshold.

Bharath Reddy Check the caption for AI interview Kit Repost

Q5. How would you design scalable vector search infrastructure?

Vector search at scale requires thinking about index architecture, hardware utilization, and operational complexity together.

→ **Index partitioning:** Do not put everything in a single index. Partition by domain (e.g., HR, Finance) and route queries appropriately to reduce search space.

→ **Read replicas for search traffic:** Vector search is read-heavy. Deploy 3-5 read replicas for query traffic; only send ingest operations to the primary.

→ **HNSW tuning:** Tune `ef_construction` and `ef_search`. A recall of 95% at P95 latency under 50ms is achievable with optimal tuning.

→ **Hardware selection:** Vector search requires high memory (RAM) because HNSW indexes fit entirely in RAM. 10M vectors (1536-dim) need ~60GB RAM.

→ **Managed services at scale:** Beyond ~50M vectors, managed databases (Pinecone, Qdrant Cloud) become operationally compelling vs self-hosting.

Bharath Reddy ↓ Check the caption for AI interview Kit Repost

Q6. When should you switch from large models to smaller models for cost optimization?

The decision to switch should be data-driven, not based on assumptions. First, measure quality baseline against an eval set representing production traffic.

Small Models

- → Simple factual extraction
- → Format conversion (JSON)
- → Classification & routing
- → Tool argument generation
- → Short summarizations

Large Models

- → Complex multi-step reasoning
- → Nuanced judgment calls
- → Long-form generation with high coherence
- → Requires broad world knowledge

Practical routing pattern: Use a classifier to route simple queries to the small model and complex queries to the large model to capture 60-70% savings while retaining quality.

Q7. How do you evaluate the cost trade-offs between self-hosted models and API models?

→ **API model costs:** `Cost = tokens × price`. No fixed costs, excellent for variable or early-stage traffic.

→ **Self-hosted costs:** GPU rental ($3-5/hr for A100), MLOps engineering, operational overhead, custom updates.

Break-even calculation: For a model at $10/1M output tokens, 100M tokens/mo = $1,000 via API.
Equivalent self-hosted: 1 A100 GPU at ~$3,000/mo. Break-even occurs roughly at 200M+ tokens/mo.

Push to Self-Hosted

→ Strict Data Privacy
→ Ultra-low Latency
→ Extensive Customization

Push to API

→ Variable peak traffic
→ Access to latest models
→ Lack of MLOps experts

Q8. How would you build dashboards for monitoring GenAI system health?

Different stakeholders need different views of system health. Build dashboards for each audience.

Engineering real-time dashboard

Request/error rate, P50/P99 latency per service, LLM API rate limits, active agent sessions, queue depth, cache hit rates.

Quality daily dashboard

Faithfulness score trends, retrieval quality metrics, task completion rates, user feedback (thumbs-down), eval set performance.

Cost weekly dashboard

Total spend by model/feature, cost per request trend, top 10 most expensive user sessions, forecast vs budget, cache savings.

Executive monthly dashboard

Total requests served/success rate, user satisfaction trend, cost per user session trend, system availability percentage.

Bharath Reddy Check the caption for AI interview Kit Repost